Software Design Description

miniaudioengine

Table of Contents

- Software Design Description
  - Table of Contents
  - 1 Introduction
  - 2 Stakeholders and Design Concerns
    - 2.1 Stakeholders
    - 2.2 Design Concerns
  - 3 Design Views
    - 3.1 Context View
    - 3.2 Composition View
      - 3.2.1 miniaudioengine SDK
      - 3.2.2 Device Manager
      - 3.2.3 File Manager
      - 3.2.4 Track Manager
    - 3.3 Logical View
      - 3.4.1 Monitor Input Device Flow
      - 3.4.2 Open File Flow
      - 3.4.3 Play to Output Device Flow
      - 3.4.4 MIDI Message Flow
      - 3.4.5 Audio Processing Flow
      - 3.4.6 Multiple Track Flow
    - 3.4 Dependency View
    - 3.5 Information View
      - 3.5.1 Devices
      - 3.5.2 Files
      - 3.5.3 Tracks
    - 3.6 Interface View
      - 3.6.1 Public SDK
      - 3.6.2 Example Programs
    - 3.7 Interaction View
      - 3.7.1 Audio Input to Audio Output
      - 3.7.3 MIDI Input Processing
    - 3.8 Structure View
      - 3.8.1 Project Structure
  - 4 Design Rationale
    - 4.1 Architectural Design

- 4.1.2 Track Hierarchy
- 4.1.3 Control Flow
- 4.1.4 Data Flow
- 4.2 External Libraries

1 Introduction

2 Stakeholders and Design Concerns

2.1 Stakeholders

Stakeholder	Responsibilities	Design Concerns
Software User	- Monitor audio input device. - Monitor MIDI input device. - Read audio files - Play audio to output device. - Play MIDI to output device.	DC-01 DC-02 DC-03 DC-04 DC-05 DC-06
Third-Party Developer	- Include SDK in C++ software application. - Handle incoming MIDI messages. - Process audio input. - Route processed audio to output device. - Manage audio in multiple tracks.	DC-07 DC-08 DC-09 DC-10 DC-11 DC-12 DC-13 DC-14 DC-17 DC-18
Maintainer	- Maintain CI/CD pipeline. - Manage code repo. - Manage software releases.	DC-13 DC-14 DC-15 DC-16 DC-17
Hardware	- Run on a Windows desktop. - Run on an embedded Linux platform.	DC-13 DC-14

2.2 Design Concerns

ID	Description	Relevant Views
DC-01	Monitor audio input device.	Logical
DC-02	Monitor MIDI input device.	Logical
DC-03	Open and read WAV audio files.	Logical
DC-04	Open and read MIDI files.	Logical
DC-05	Route audio to output device.	Logical
DC-06	Route MIDI to output device.	Logical
DC-07	Processing incoming MIDI messages.	Logical
DC-08	Processing incoming audio streams.	Logical
DC-09	Manage multiple audio tracks.	Logical
DC-10	Add one audio or MIDI input to a track.	Logical
DC-11	Attach one audio or MIDI output to a track.	Logical
DC-12	Chain multiple audio processors in one track.	Logical
DC-13	Build software SDK on Windows and Linux.	Context

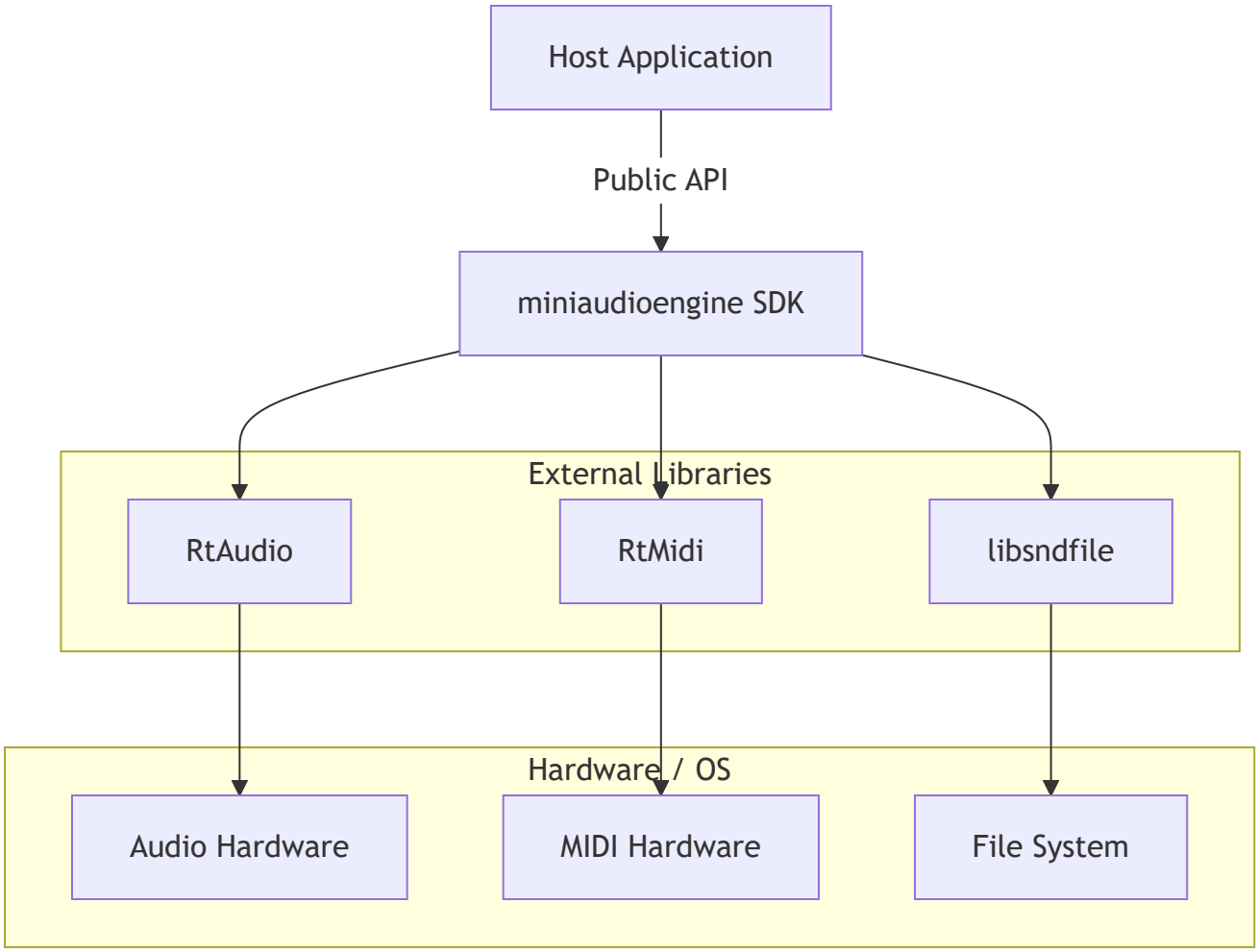
ID	Description	Relevant Views
DC-14	Build software SDK on x86_64 and ARM64 platforms.	
DC-15	CI/CD pipeline builds and packages software on all compatible platforms.	
DC-16	Complete unit testing and code coverage.	
DC-17	Package software as an SDK used by third-party software.	Context, Composition
DC-18	Third-party software developers manage audio tracks, system audio/MIDI devices, and filesystem.	Composition

3 Design Views

3.1 Context View

Describes the software in context with its external environment. Define users, external components, and the interactions between.

Design Concern	
DC-13	Build software SDK on Windows and Linux.
DC-17	Package software as an SDK used by third-party software.

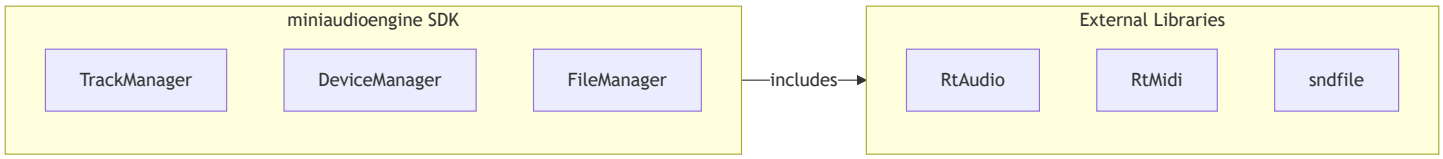


3.2 Composition View

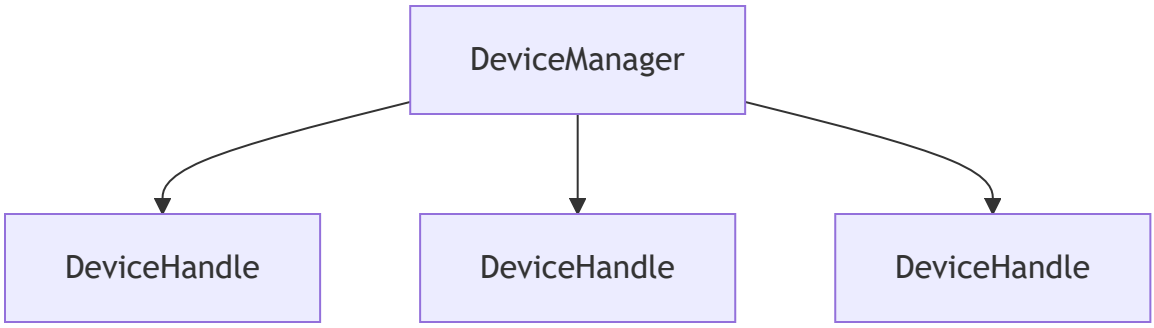
Describe the composition of the **miniaudioengine** SDK software libraries.

Design Concern	
DC-17	Package software as an SDK used by third-party software.
DC-18	Third-party software developers manage audio tracks, system audio/MIDI devices, and filesystem.

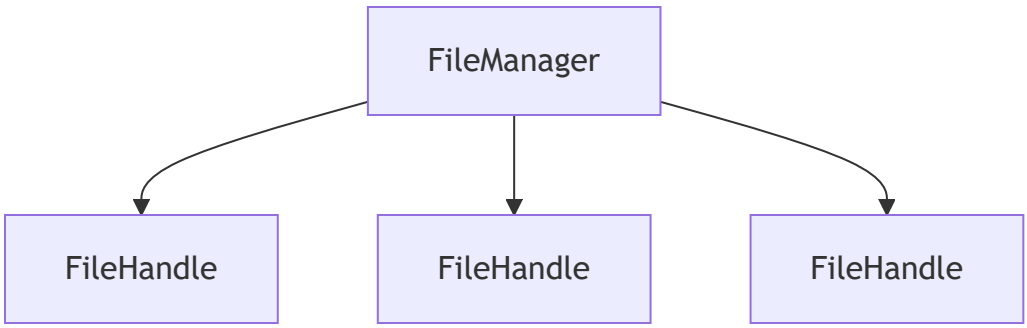
3.2.1 miniaudioengine SDK



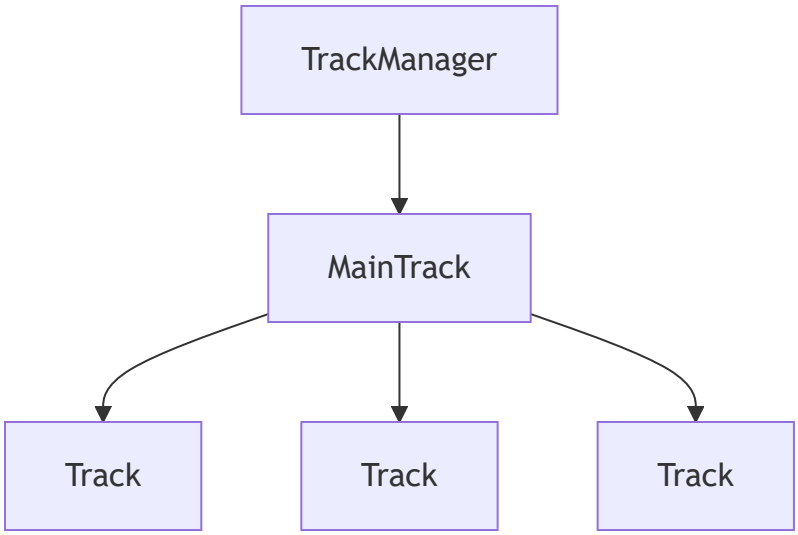
3.2.2 Device Manager



3.2.3 File Manager



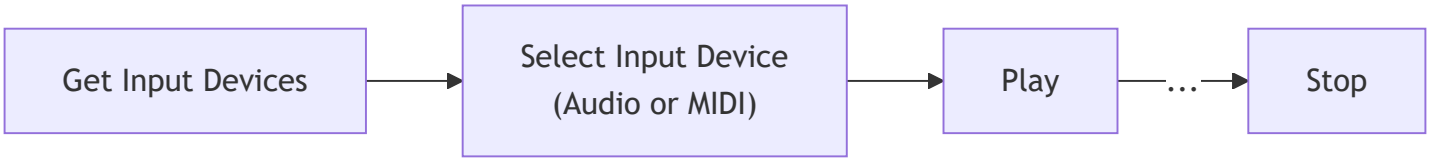
3.2.4 Track Manager



3.3 Logical View

3.4.1 Monitor Input Device Flow

Design Concern	
DC-01	Monitor audio input device.
DC-02	Monitor MIDI input device.

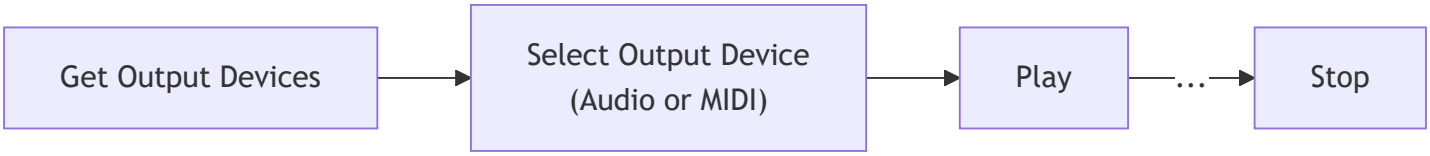


3.4.2 Open File Flow

Design Concern	
DC-03	Open and read WAV audio files.
DC-04	Open and read MIDI files.

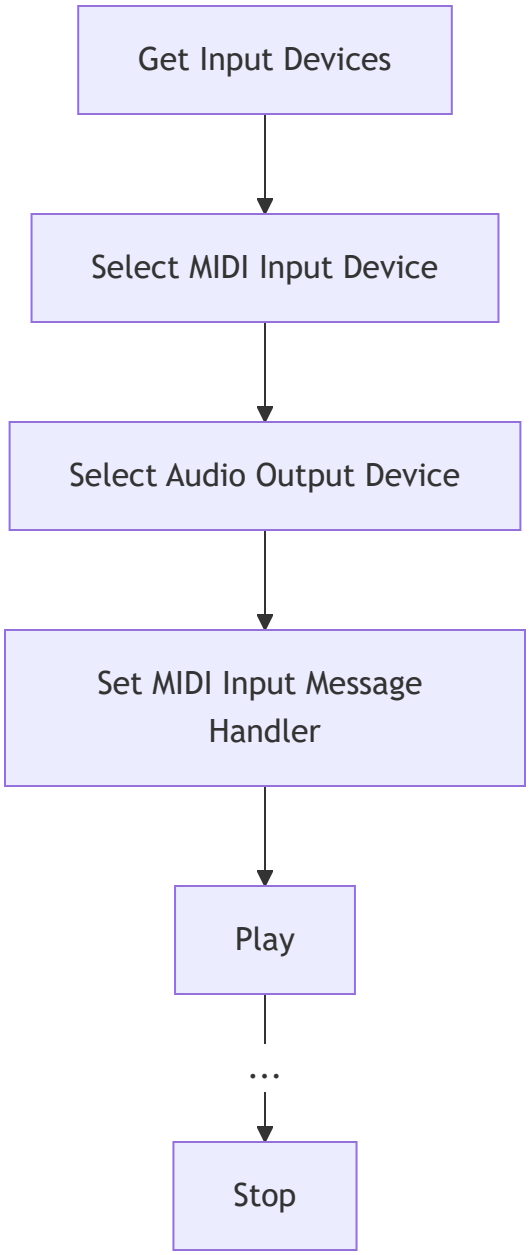
3.4.3 Play to Output Device Flow

Design Concern	
DC-05	Route audio to output device.
DC-06	Route MIDI to output device.



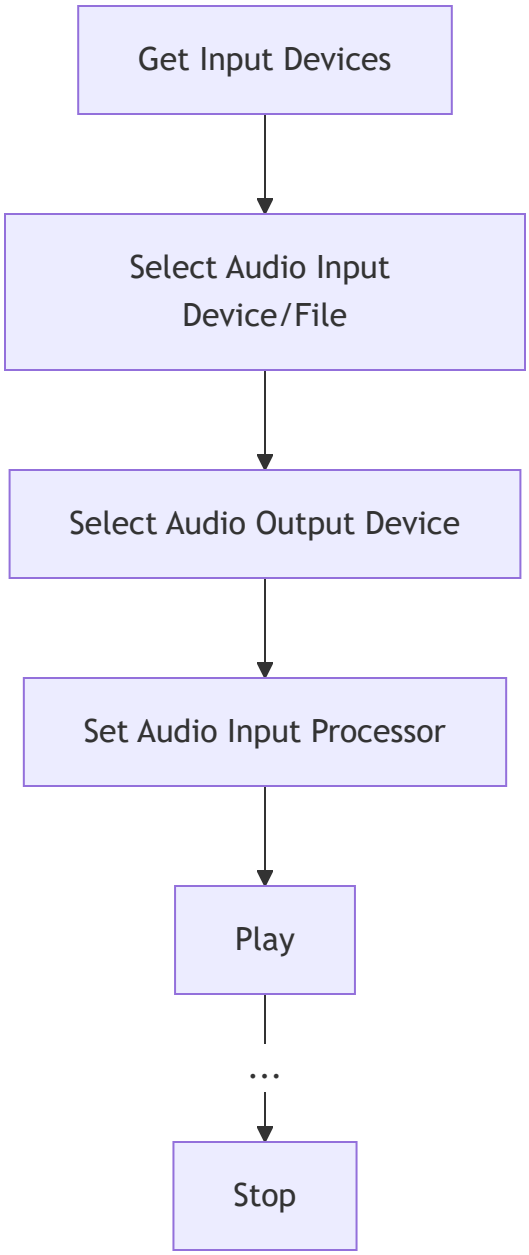
3.4.4 MIDI Message Flow

Design Concern	
DC-07	Processing incoming MIDI messages.



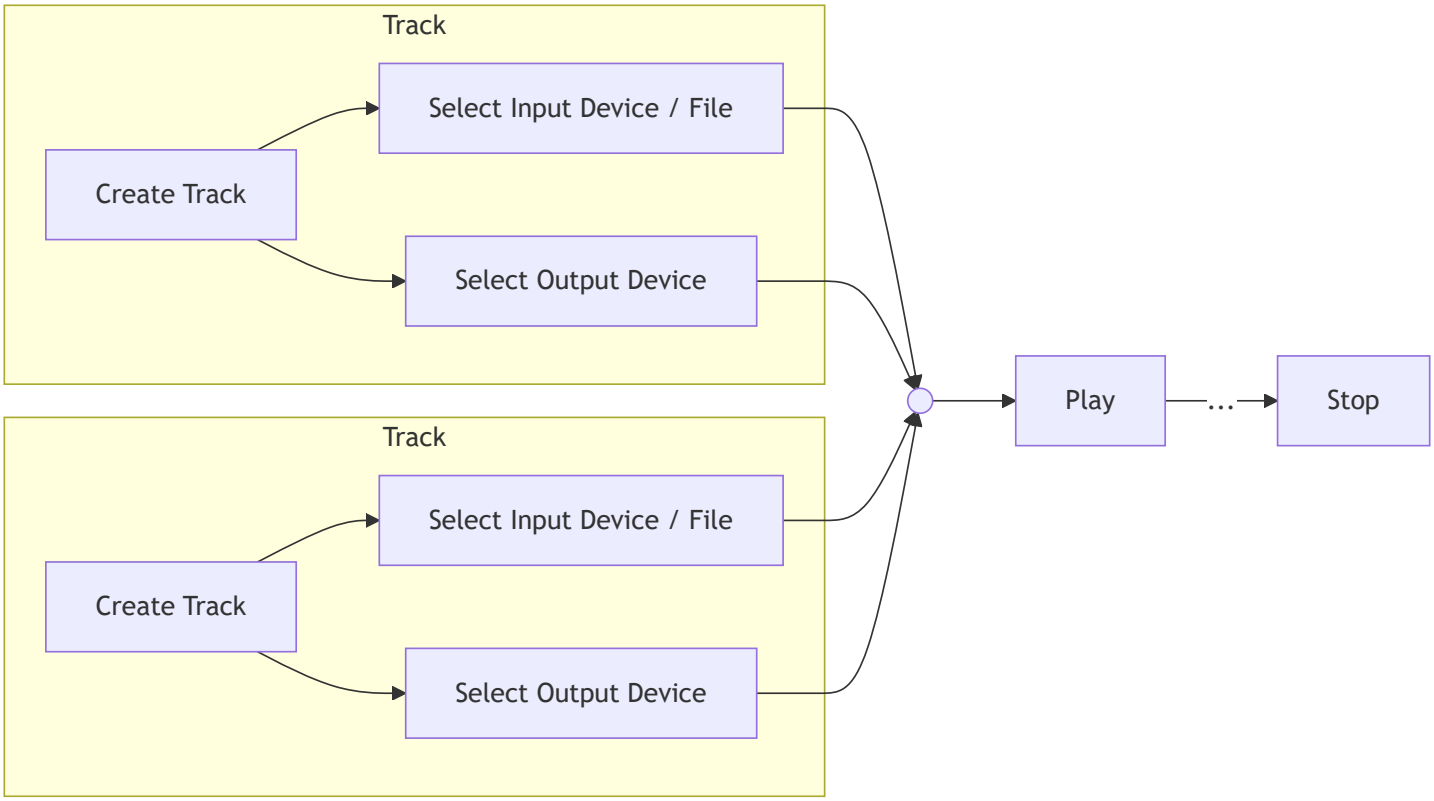
3.4.5 Audio Processing Flow

Design Concern	
DC-08	Processing incoming audio streams.



3.4.6 Multiple Track Flow

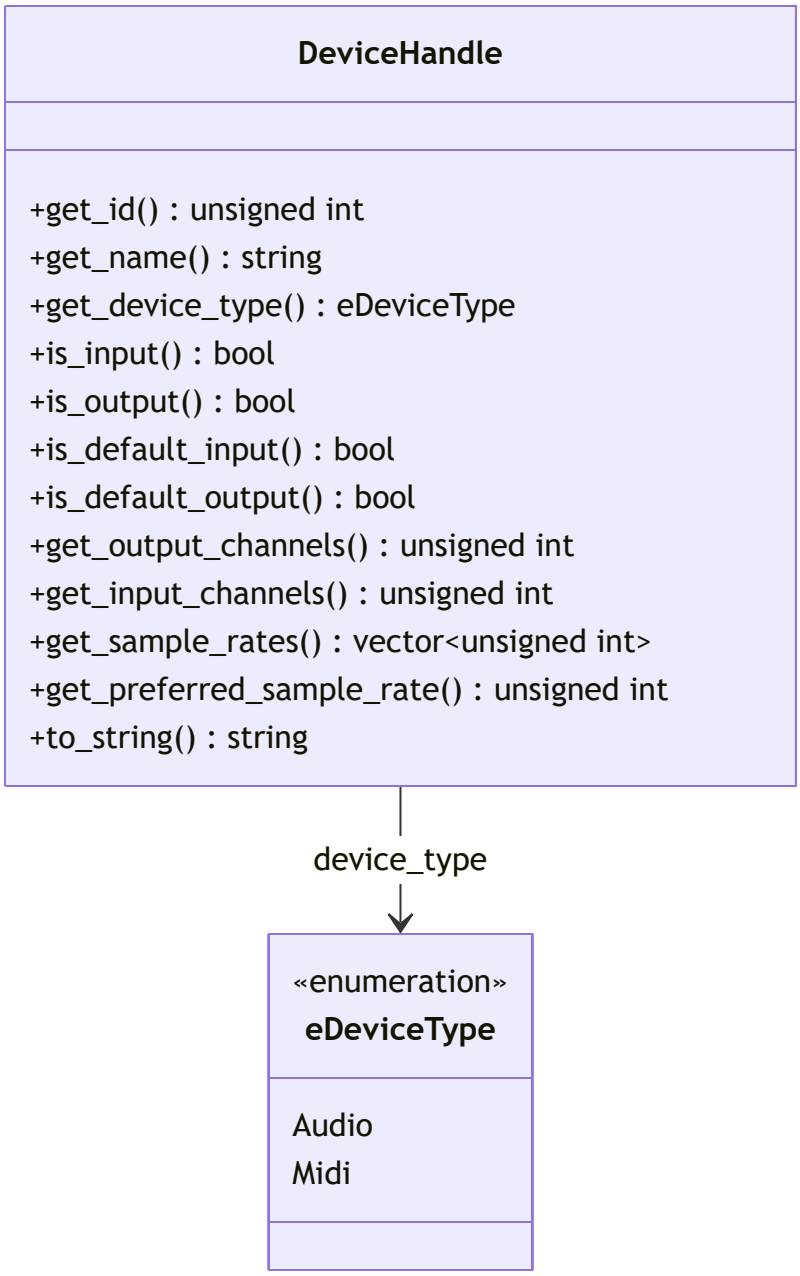
Design Concern	
DC-09	Manage multiple audio tracks.
DC-10	Add one audio or MIDI input to a track.
DC-11	Attach one audio or MIDI output to a track.
DC-12	Chain multiple audio processors in one track.



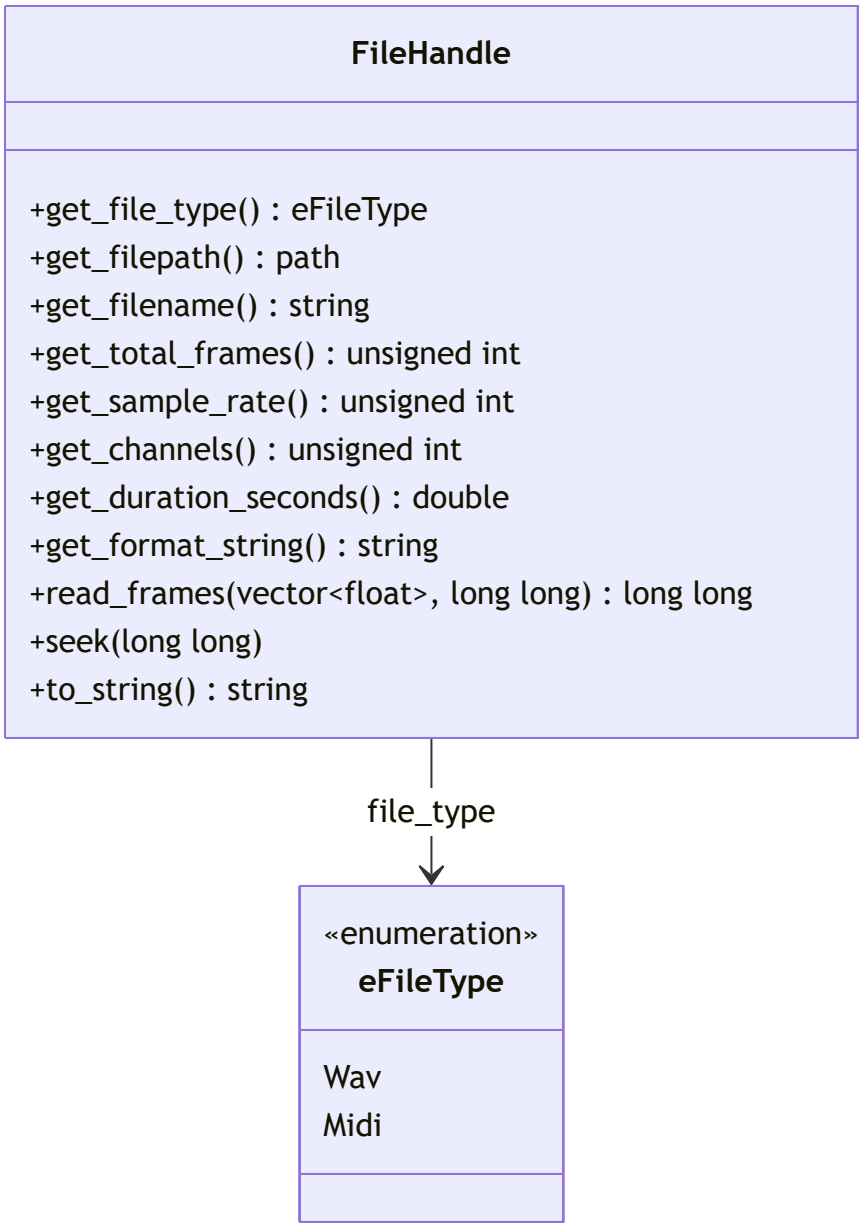
3.4 Dependency View

3.5 Information View

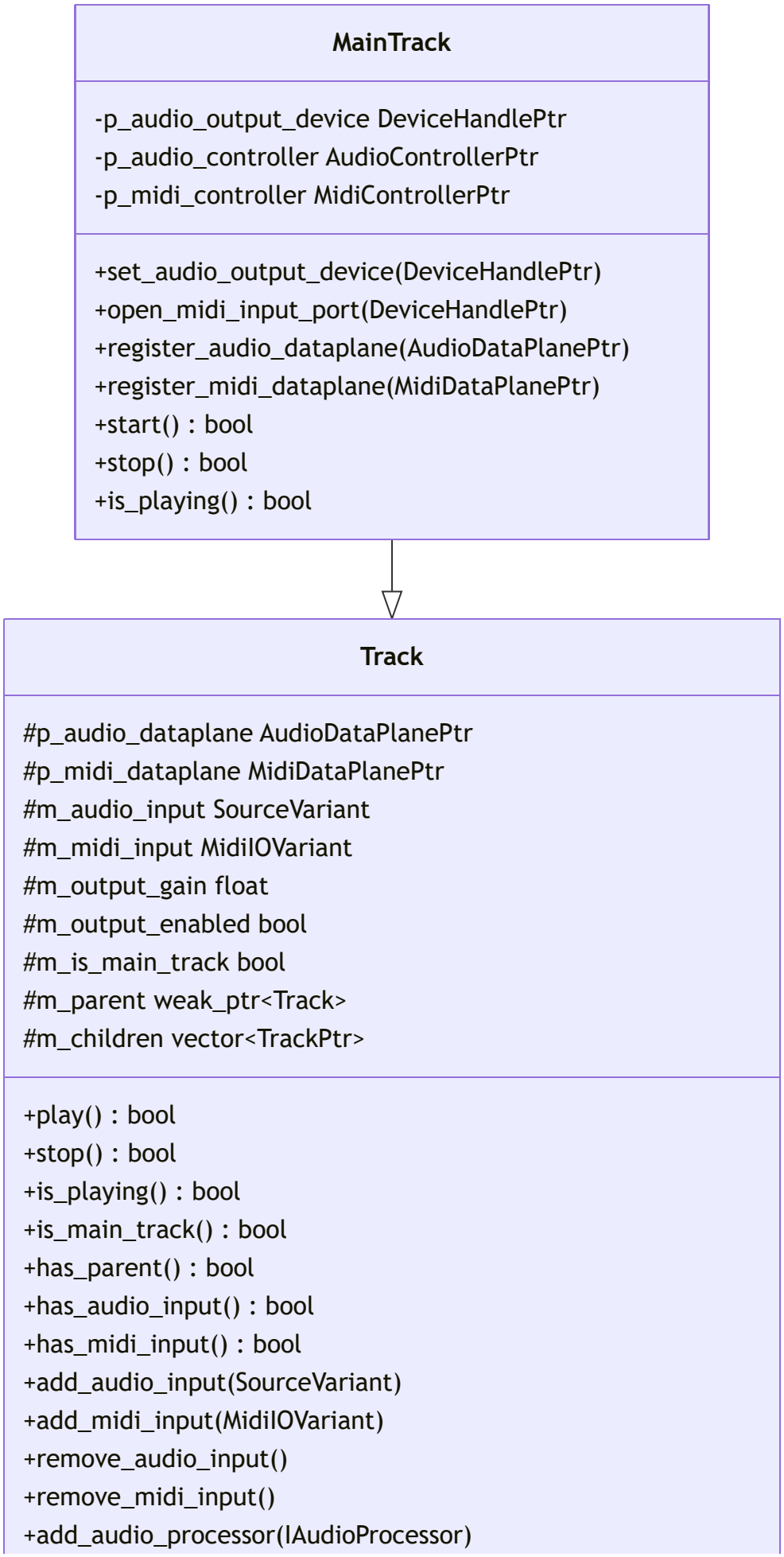
3.5.1 Devices



3.5.2 Files


FileHandle
+get_file_type() : eFileType
+get_filepath() : path
+get_filename() : string
+get_total_frames() : unsigned int
+get_sample_rate() : unsigned int
+get_channels() : unsigned int
+get_duration_seconds() : double
+get_format_string() : string
+read_frames(vector<float>, long long) : long long
+seek(long long)
+to_string() : string
file_type
«enumeration»
eFileType
Wav
Midi

3.5.3 Tracks


MainTrack
-p_audio_output_device DeviceHandlePtr
-p_audio_controller AudioControllerPtr
-p_midi_controller MidiControllerPtr
+set_audio_output_device(DeviceHandlePtr)
+open_midi_input_port(DeviceHandlePtr)
+register_audio_dataplane(AudioDataPlanePtr)
+register_midi_dataplane(MidiDataPlanePtr)
+start() : bool
+stop() : bool
+is_playing() : bool
Track
#p_audio_dataplane AudioDataPlanePtr
#p_midi_dataplane MidiDataPlanePtr
#m_audio_input SourceVariant
#m_midi_input MidiIOVariant
#m_output_gain float
#m_output_enabled bool
#m_is_main_track bool
#m_parent weak_ptr<Track>
#m_children vector<TrackPtr>
+play() : bool
+stop() : bool
+is_playing() : bool
+is_main_track() : bool
+has_parent() : bool
+has_audio_input() : bool
+has_midi_input() : bool
+add_audio_input(SourceVariant)
+add_midi_input(MidiIOVariant)
+remove_audio_input()
+remove_midi_input()
+add_audio_processor(IAudioProcessor)

+add_child_track(TrackPtr)
+remove_child_track(TrackPtr)
+get_parent() : TrackPtr
+get_children() : vector<TrackPtr>
+set_output_gain(float)
+get_output_gain() : float
+enable_output(bool)
+is_output_enabled() : bool
+get_audio_dataplane() : AudioDataPlanePtr
+get_midi_dataplane() : MidiDataPlanePtr
+get_statistics() : TrackStatistics
+set_event_callback(TrackEventCallback)
+set_midi_note_on_callback(MidiNoteOnCallbackFunc)
+set_midi_note_off_callback(MidiNoteOffCallbackFunc)
+set_midi_control_change_callback(MidiControlCallbackFunc)
«enumeration»
eTrackEvent
PlaybackFinished

3.6 Interface View

3.6.1 Public SDK

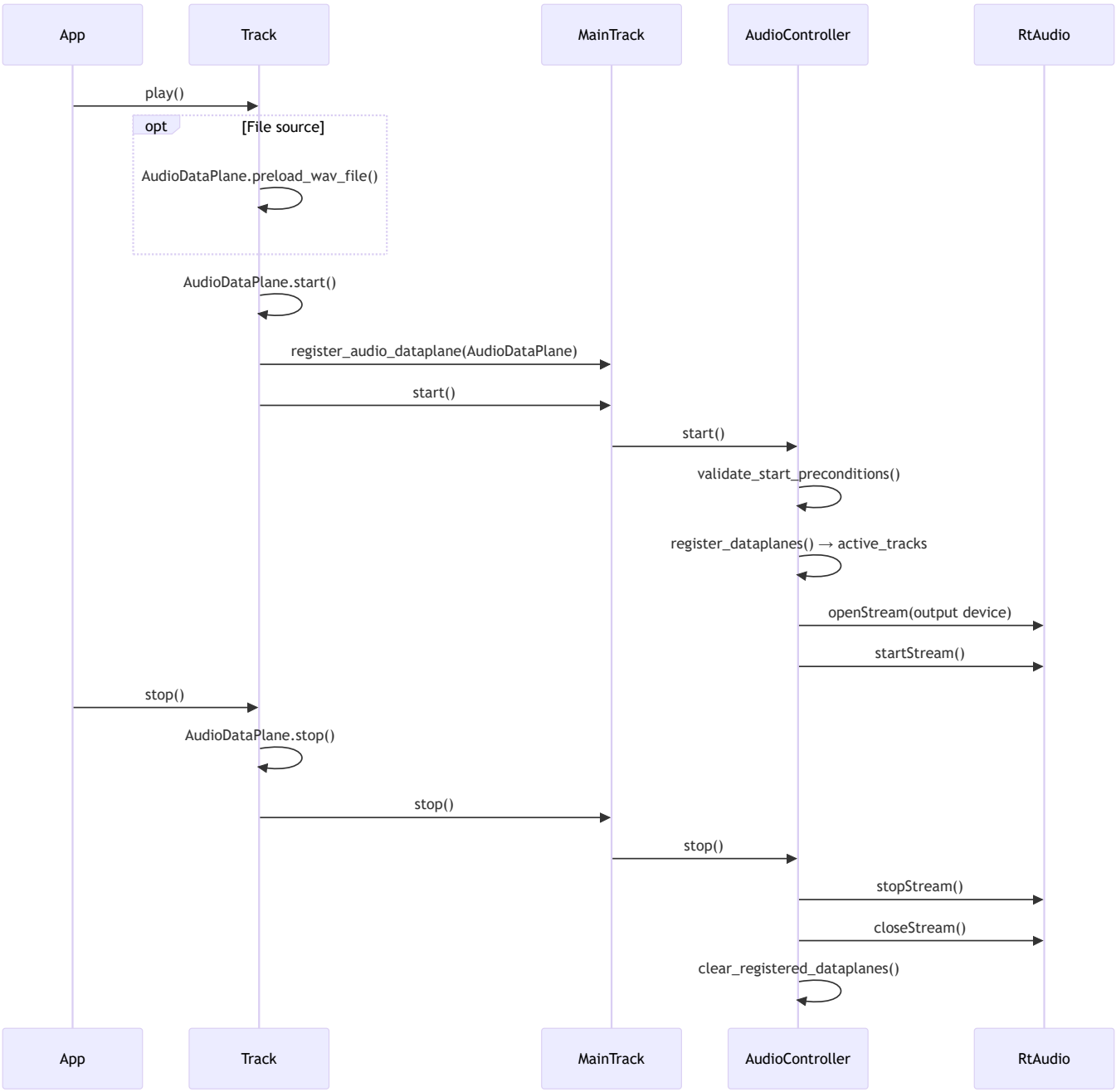
```
// miniaudioengine.h

// Types
using DeviceHandlePtr = std::shared_ptr<DeviceHandle>;
using FileHandlePtr = std::shared_ptr<FileHandle>;
using TrackPtr = std::shared_ptr<Track>;

// Playback
bool miniaudioengine::play();
bool miniaudioengine::record();
bool miniaudioengine::stop();
bool miniaudioengine::is_running();

// Devices
std::shared_ptr<DeviceManager> miniaudioengine::get_device_manager();
std::vector<DeviceHandlePtr> miniaudioengine::get_audio_devices();
std::vector<DeviceHandlePtr> miniaudioengine::get_midi_devices();
bool miniaudioengine::set_output_device(DeviceHandlePtr device);
bool miniaudioengine::set_input_device(DeviceHandlePtr device);

// Files
std::shared_ptr<FileManager> miniaudioengine::get_file_manager();
std::vector<FileHandlePtr> miniaudioengine::get_audio_files(const std::filesystem::path &directo
std::vector<FileHandlePtr> miniaudioengine::get_wav_files(const std::filesystem::path &directory
bool miniaudioengine::set_input_file(FileHandlePtr device);

// Tracks
std::shared_ptr<TrackManager> miniaudioengine::get_track_manager();
std::vector<TrackPtr> miniaudioengine::get_tracks();
TrackPtr miniaudioengine::add_track();
bool miniaudioengine::remove_track(TrackPtr track);
bool miniaudioengine::clear_tracks();
```

3.6.2 Example Programs

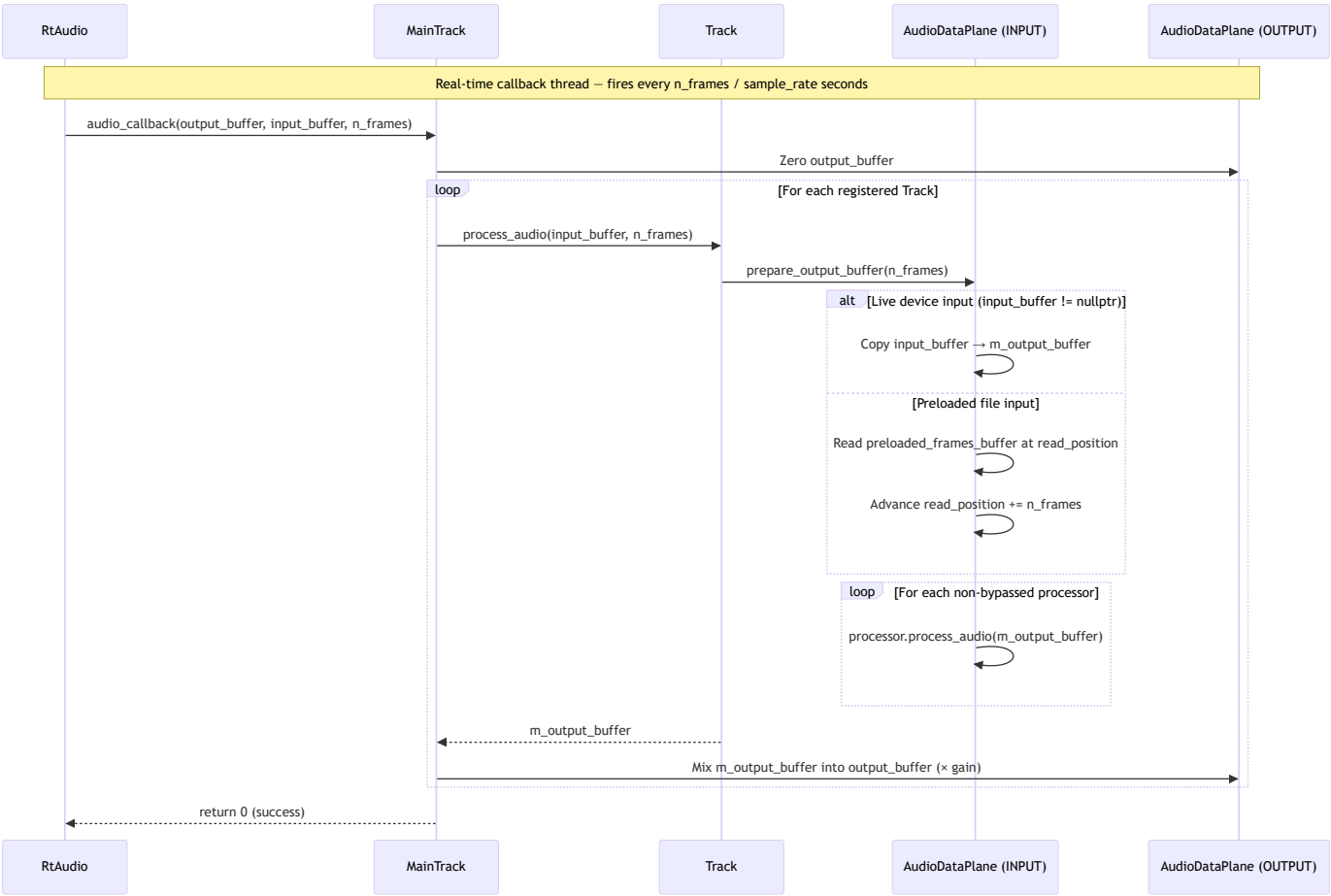
3.7 Interaction View

3.7.1 Audio Input to Audio Output

Control Plane

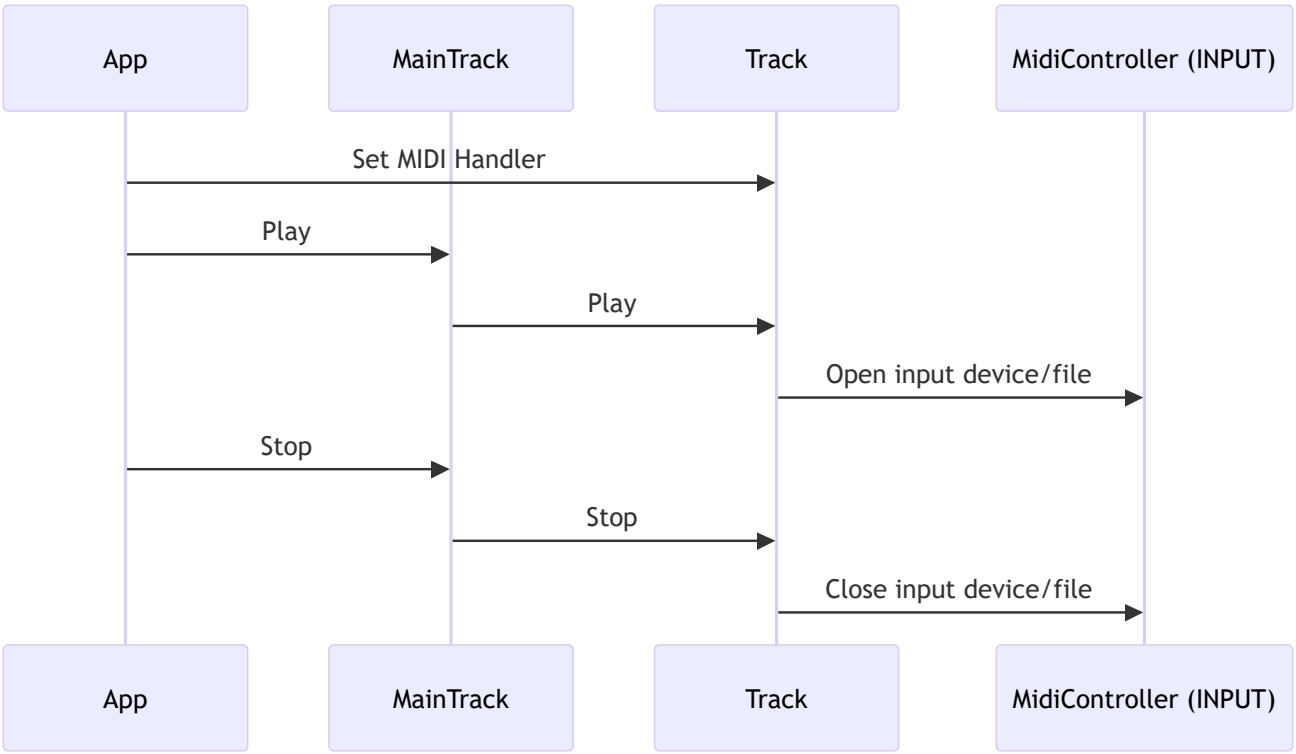


Data Plane


RtAudio
MainTrack
Track
AudioDataPlane (INPUT)
AudioDataPlane (OUTPUT)
Real-time callback thread — fires every n_frames / sample_rate seconds
audio_callback(output_buffer, input_buffer, n_frames)
Zero output_buffer
loop
[For each registered Track]
process_audio(input_buffer, n_frames)
prepare_output_buffer(n_frames)
alt
[Live device input (input_buffer != nullptr)]
Copy input_buffer → m_output_buffer
[Preloaded file input]
Read preloaded_frames_buffer at read_position
Advance read_position += n_frames
loop
[For each non-bypassed processor]
processor.process_audio(m_output_buffer)
m_output_buffer
Mix m_output_buffer into output_buffer (× gain)
return 0 (success)
RtAudio
MainTrack
Track
AudioDataPlane (INPUT)
AudioDataPlane (OUTPUT)

3.7.3 MIDI Input Processing

Control Plane



3.8 Structure View

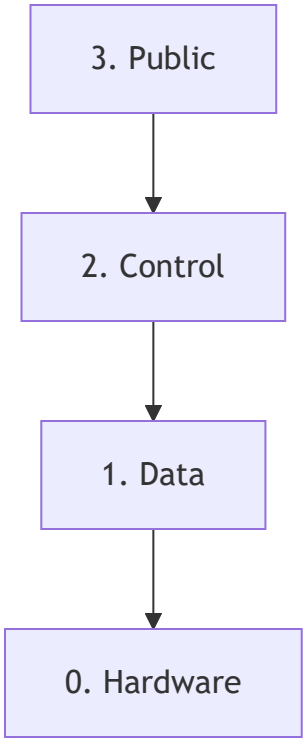
3.8.1 Project Structure

```
cmake/
    miniaudioengine-config.cmake.in
docker/
    docker-build.sh
    docker-run.sh
    docker-setup.sh
    docker-exec.sh
docs/
examples/                   # Example programs using miniaudioengine SDK
samples/
include/
    miniaudioengine/
        miniaudioengine.h   # Public facing SDK
src/
    framework/              # Internal, core shared library
        audio/
        midi/
    devices/
    files/
    tracks/
tests/
CMakeLists.txt              # CMake instructions
CMakePresets.json           # CMake presets
Dockerfile                  # Build Docker image for build environment
vcpkg.json                  # Windows VCPKG dependencies
```

4 Design Rationale

4.1 Architectural Design

4.1.1 Layered Architecture



4.1.2 Track Hierarchy

4.2 External Libraries